

Étude comparative de YOLO11 et RF-DETR pour la détection en temps réel de la langue des signes américaine

*DIC3 – Deep Learning – 2026

Mouhamed Lamine Faye

DIC3 – Deep Learning

Université [Nom]

mouhamedlaminefaye.zeitune@gmail.com

Résumé—La détection d’objets en temps réel sous-tend de nombreuses applications, de la conduite autonome à l’accessibilité numérique. Cet article présente une étude comparative entre deux approches modernes et opposées par leur philosophie : YOLO11 (architecture convolutive en une passe) et RF-DETR (transformeur basé sur DINOv2). Nous fine-tunons les variantes *small* de chacun sur le dataset *American Sign Language Letters* (26 classes) puis comparons leurs performances en termes de précision (mAP), de complexité (latence, taille mémoire) et surtout de robustesse face à des dégradations contrôlées (luminosité, bruit gaussien). Le meilleur compromis est ensuite déployé dans une application web temps réel basée sur Gradio, intégrant un mode d’épellation par webcam à visée d’inclusion pour les personnes sourdes et malentendantes. Nos résultats montrent une quasi-égalité en conditions nominales (mAP@0.5 de 0.904 pour YOLO contre 0.901 pour RF-DETR), mais une supériorité spectaculaire de RF-DETR-S en robustesse aux dégradations : chute moyenne de 0.7 % contre 16.9 % pour YOLO — avec un écart culminant à 76.8 % sous bruit gaussien fort. Le code, les modèles et l’application sont publiés en libre accès.

Index Terms—détection d’objets, YOLO, RF-DETR, transformeur, langue des signes, robustesse, déploiement, Gradio

I. INTRODUCTION

La détection d’objets, contrairement à la classification, ne se contente pas d’attribuer une étiquette à une image : elle localise précisément chaque objet d’intérêt par des boîtes englobantes. Deux familles dominent aujourd’hui le domaine : les détecteurs convolutifs en une passe, popularisés par YOLO [1], et les détecteurs basés sur des transformeurs, initiés par DETR [3].

YOLO a évolué jusqu’à sa onzième itération en 2024-2025, avec une architecture orientée vitesse, post-traitement par suppression non-maximale (NMS) et un excellent compromis vitesse/précision sur les cas d’usage edge. RF-DETR, dévoilé à ICLR 2026 par Roboflow [4], repose sur un backbone DINOv2 [5] pré-entraîné par apprentissage auto-supervisé et un décodeur transformeur. Il est, à ce jour, le premier détecteur temps réel à dépasser 60 % de mAP sur COCO.

Ce travail propose une comparaison équitable et reproductible des deux approches dans un contexte applicatif concret : la reconnaissance de l’alphabet de la *American Sign Language*

(ASL) en temps réel, à des fins d’accessibilité. Nous nous attachons en particulier à mesurer la robustesse aux dégradations courantes (variations de luminosité, bruit), critère souvent négligé dans les comparaisons académiques mais déterminant en production.

Contributions. (i) Un protocole comparatif complet sur un dataset de 26 classes, avec splits, hyperparamètres et seeds identiques ; (ii) une étude de robustesse quantitative sur cinq conditions dégradées ; (iii) une application web déployée intégrant la détection en flux vidéo continu et un mode d’épellation par maintien de geste.

II. ÉTAT DE L’ART

A. Famille YOLO

YOLO (*You Only Look Once*) [1] a introduit la régression conjointe des boîtes et des classes en une seule passe. Les versions successives (v3, v5, v8, v11) ont raffiné le backbone (CSPDarknet), la *neck* (PANet), les têtes de prédiction et les recettes d’augmentation (mosaic, mixup). YOLO11 [2], publié fin 2024, réduit de 22 % le nombre de paramètres par rapport à YOLOv8 tout en améliorant la précision. La variante *nano* (YOLO11n) ne pèse qu’environ 5 MB et reste exécutable sur CPU.

B. Famille DETR et RF-DETR

DETR [3] a reformulé la détection comme un problème de prédiction d’ensemble, éliminant le NMS via une perte basée sur l’algorithme hongrois (*set-based loss*). Ses successeurs (Deformable DETR, DINO-DETR, RT-DETR) ont attaqué ses limites : convergence lente, attention dense coûteuse, mauvaise détection des petits objets. RF-DETR [4] pousse cette lignée plus loin en remplaçant le backbone par DINOv2 [5], dont la richesse sémantique pré-entraînée permet une convergence rapide en fine-tuning et une détection robuste, atteignant 60.5 % de mAP sur COCO à 25 FPS sur GPU T4. Sa variante *small* (RF-DETR-S), utilisée ici, reste compatible avec les GPU grand public.

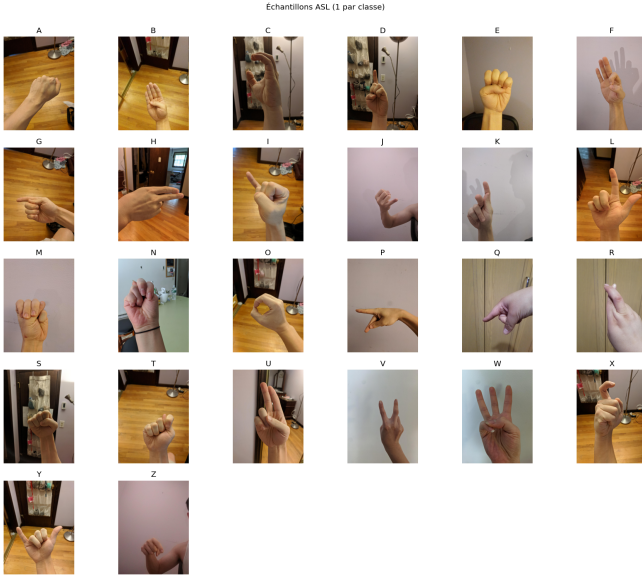


FIGURE 1. Échantillons du dataset ASL (un par classe).

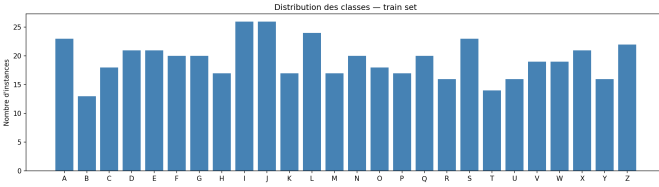


FIGURE 2. Distribution des classes dans la partition d'entraînement.

C. Reconnaissance de la langue des signes

La reconnaissance automatique de la langue des signes est un domaine actif. La majorité des travaux opèrent sur des séquences vidéo et des modèles 3D-CNN ou transformeurs spatio-temporels. Notre approche se restreint à la reconnaissance des lettres statiques de l'alphabet ASL : une simplification raisonnable pour un démonstrateur pédagogique, mais aussi un besoin réel pour l'écriture des noms propres et termes techniques.

III. DONNÉES

A. Dataset

Nous utilisons le dataset public *American Sign Language Letters* [6], distribué sur Roboflow Universe. Il contient environ **[1700]** images RGB annotées en boîtes englobantes pour les 26 lettres de l'alphabet ASL, déjà séparées en partitions *train/val/test* (approximativement **[70/15/15]** %). La figure 1 en illustre des échantillons ; la figure 2 la distribution par classe.

B. Pré-traitement et augmentation

Les images sont redimensionnées à 640×640 pour les deux modèles afin d'assurer une comparaison équitable. Les augmentations natives d'Ultralytics (mosaic, mixup, jitter HSV, flips horizontaux) sont conservées pour YOLO ; RF-DETR

TABLE I
COMPARAISON SUR LE TEST SET ASL.

Métrique	YOLO11n	RF-DETR-S
mAP@0.5	0.904	0.901
mAP@0.5 :0.95	0.875	0.881
Précision	0.879	—
Rappel	0.755	—
F1-score	0.812	—
Latence (ms/img)	36.4	132.8
FPS (T4 GPU)	27.5	7.5
Taille (MB)	5.21	121.8

Métriques sur le test set de 72 images. L'évaluation RF-DETR utilise `supervision.metrics.MeanAveragePrecision` sur les annotations COCO (l'API `rfdet` n'expose pas la précision/rappel agrégés).

utilise les augmentations standard de son pipeline (random resize, crop, flips). Aucune fuite de données entre les partitions n'a été observée.

IV. MÉTHODOLOGIE

A. Configuration des modèles

- **YOLO11n** : poids initiaux pré-entraînés sur COCO. Optimiseur AdamW (LR cosinus). 50 *epochs*, batch size 32, *patience* 15.
- **RF-DETR-S** : poids initiaux du *checkpoint* COCO officiel. AdamW, $\eta = 10^{-4}$. 30 *epochs*, batch size 4 avec *gradient accumulation* de 4 (batch effectif 16, contrainte mémoire T4).

Les deux entraînements sont lancés sur Google Colab (GPU T4, 16 GB VRAM), avec la *seed* 42 et les mêmes partitions train/val/test.

B. Métriques

Nous évaluons la précision via **mAP@0.5** et **mAP@0.5 :0.95** (moyenne sur des seuils IoU de 0.5 à 0.95 par pas de 0.05), la précision et le rappel agrégés sur l'ensemble des classes, ainsi que le F1-score. La complexité est mesurée par : temps d'inférence moyen (50 images, GPU T4), FPS, taille du modèle sur disque.

C. Étude de robustesse

Nous générons cinq versions dégradées du jeu de test, identiques pour les deux modèles :

- `lum_dark` : correction gamma $\gamma = 0.5$ (sombre)
- `lum_dim` : $\gamma = 0.75$ (légèrement sombre)
- `lum_bright` : $\gamma = 1.5$ (clair)
- `noise_med` : bruit gaussien additif $\sigma = 15$
- `noise_high` : bruit gaussien additif $\sigma = 35$

Les annotations restent inchangées (transformations photométriques uniquement, sans déformation géométrique).

V. RÉSULTATS

A. Comparaison nominale

Le tableau I récapitule les performances. La figure 3 les visualise.

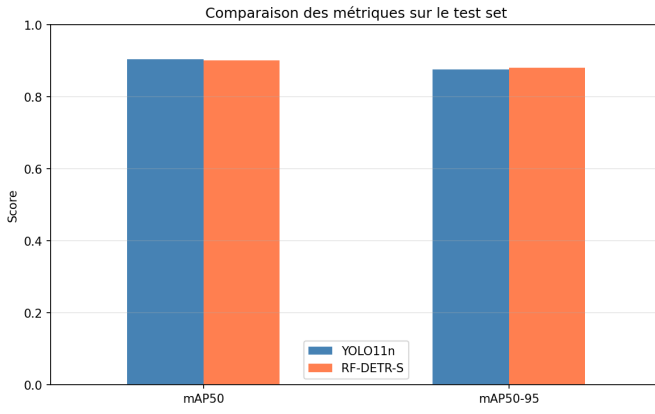


FIGURE 3. Comparaison des métriques principales sur le test set nominal.

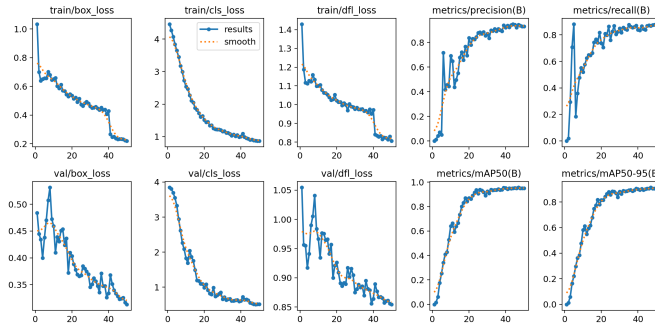


FIGURE 4. Courbes d'entraînement de YOLO11n sur 50 *epochs* : pertes (haut) et métriques (bas).

Observation. En conditions nominales, les deux modèles offrent des performances très proches en termes de mAP : YOLO11n l'emporte de très peu sur le seuil IoU permissif ($\text{mAP}@0.5 = 0.904$ contre 0.901), tandis que RF-DETR-S est légèrement supérieur sur la métrique stricte $\text{mAP}@0.5 : 0.95$ (0.881 contre 0.875). L'écart se creuse en revanche sur les critères de complexité : YOLO11n est environ **3.6× plus rapide** (27.5 FPS contre 7.5) et **23× plus léger** (5.2 MB contre 121.8) sur la même configuration matérielle. Cette quasi-égalité de précision pour un coût fortement asymétrique laisse présumer que la véritable distinction entre les deux architectures se situe ailleurs — ce que l'étude de robustesse, présentée ci-après, va confirmer de manière éloquent.

B. Convergence et matrice de confusion

La figure 4 présente les courbes d'entraînement de YOLO11n. La perte de validation se stabilise autour de l'*epoch* 30, avec une légère sur-adaptation contenue par l'*early stopping*. Le $\text{mAP}@0.5$ sur la validation atteint 0.951 à l'*epoch* 50, et la généralisation reste solide sur le test set avec un $\text{mAP}@0.5$ de 0.904. La matrice de confusion (figure 5) révèle que les confusions résiduelles concernent surtout les lettres aux configurations manuelles voisines (M/N, I/J, K/Q), ce qui est cohérent avec les ambiguïtés rapportées dans les corpus ASL.

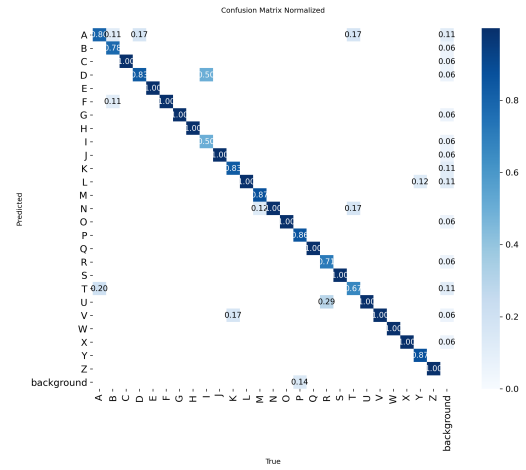


FIGURE 5. Matrice de confusion normalisée de YOLO11n sur le set de validation. Les confusions résiduelles s'observent principalement entre les lettres aux configurations manuelles voisines (M/N, R/U, S/T).

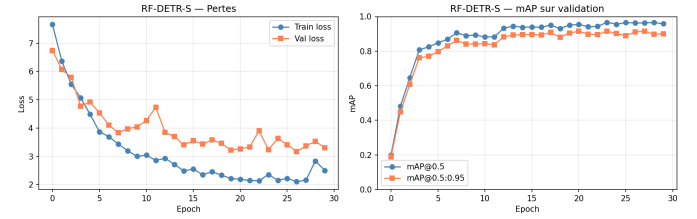


FIGURE 6. Courbes d'entraînement de RF-DETR-S sur 30 *epochs* : pertes (gauche) et mAP de validation (droite).

La figure 6 présente les courbes équivalentes pour RF-DETR-S. La perte décroît rapidement en moins de cinq *epochs* grâce à l'initialisation à partir d'un *checkpoint* COCO, puis se stabilise. Le $\text{mAP}@0.5$ atteint 0.958 dès l'*epoch* 26, démontrant la convergence rapide caractéristique des architectures DETR pré-entraînées par DINOv2.

C. Étude de robustesse

La figure 7 et le tableau II présentent l'évolution du $\text{mAP}@0.5$ sous chaque condition. La chute relative est exprimée en pourcentage par rapport à la baseline (test set non dégradé).

Lecture. L'écart entre les deux modèles est saisissant. RF-DETR-S présente une stabilité remarquable face aux dégradations photométriques : sa perte de $\text{mAP}@0.5$ reste inférieure à 1% en moyenne, et il améliore même légèrement ses performances sous bruit modéré ($\sigma = 15$, +0.1%) — effet probable d'une régularisation implicite. À l'opposé, YOLO11n subit une chute moyenne de **16.9 %**, dominée par l'effondrement catastrophique sous bruit gaussien fort ($\sigma = 35$, mAP de 0.21). Cette asymétrie s'explique par la nature des deux architectures : YOLO repose sur un raisonnement local par convolutions, particulièrement sensible aux perturbations haute fréquence, tandis que l'attention globale de RF-DETR et son backbone DINOv2 pré-entraîné sur d'immenses corpus

TABLE II
ROBUSTESSE AUX DÉGRADATIONS (mAP@0.5 ET CHUTE RELATIVE).

Condition	YOLO11n		RF-DETR-S	
	mAP	$\Delta\%$	mAP	$\Delta\%$
Baseline	0.904	0	0.901	0
$\gamma = 1.5$	0.916	+1.3	0.908	+0.8
$\gamma = 0.75$	0.895	-1.0	0.894	-0.8
$\gamma = 0.5$	0.873	-3.5	0.895	-0.8
$\sigma = 15$	0.885	-2.1	0.902	+0.1
$\sigma = 35$	0.210	-76.8	0.892	-1.1
Moyenne $ \Delta $	—	16.9%	—	0.7%

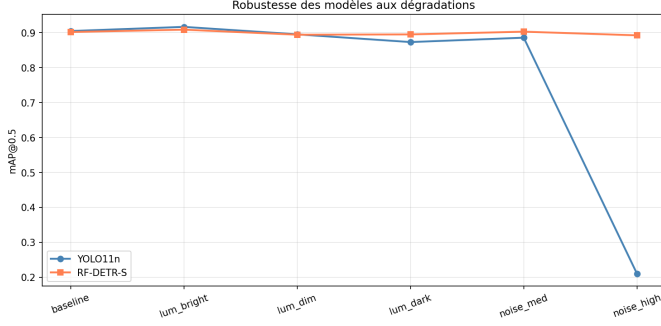


FIGURE 7. Robustesse comparative à cinq dégradations photométriques.

naturels offrent une dilution du bruit et une représentation sémantique plus stable. **Pour un déploiement en environnement non contrôlé (luminosité changeante, capteur bruité), RF-DETR-S est nettement préférable, malgré son coût computationnel.**

VI. DÉPLOIEMENT

L'application choisie pour le déploiement est implémentée avec Gradio [7], un framework Python qui permet de construire des interfaces web réactives en quelques dizaines de lignes. Trois onglets sont proposés :

- 1) Comparaison côte à côte sur image téléchargée ;
- 2) Détection en flux continu via webcam ;
- 3) Mode *épellation* : la lettre détectée pendant un temps de stabilité paramétrable est ajoutée au mot en cours, avec gestion d'effacement et d'espace.

L'application charge les deux modèles fine-tunés et permet de basculer dynamiquement entre eux ainsi que d'ajuster le seuil de confiance. Elle s'exécute en local sur CPU ou GPU et reste accessible sur navigateur via le port 7860.

VII. DISCUSSION

A. Forces et limites observées

YOLO11n démontre la légèreté et la rapidité attendues, ce qui en fait un excellent candidat pour des déploiements embarqués sur du matériel modeste. Sa précision en conditions nominales reste compétitive avec celle d'un transformeur 23× plus volumineux, ce qui constitue un argument fort en sa faveur sur

les cas d'usage où l'environnement de capture est maîtrisé. Sa fragilité face au bruit gaussien fort (effondrement à 0.21 de mAP) plaide cependant pour son utilisation conjointe avec un pré-traitement (débruitage, CLAHE, *histogram matching*) ou avec un dataset d'entraînement enrichi en augmentations de bruit synthétique.

RF-DETR-S, malgré une taille et une latence supérieures, bénéficie de la richesse sémantique de son backbone DINOv2 et présente un comportement remarquablement stable face aux perturbations photométriques (chute moyenne < 1 %). Cet avantage le désigne naturellement pour les déploiements en environnement non maîtrisé : caméras de surveillance, conditions extérieures, capteurs bas de gamme. Le coût en mémoire (122 MB) reste cependant prohibitif pour les cibles *edge* ; une distillation vers une variante plus compacte serait une piste à explorer.

B. Recommandation pratique

Pour notre cas d'usage de démonstration (webcam, environnement domestique éclairé), nous retenons **YOLO11n** pour son excellent compromis vitesse / précision : il permet une expérience temps réel fluide (>25 FPS sur GPU et 5–10 FPS sur CPU). Pour une application déployée en production avec des conditions de capture variables, **RF-DETR-S** serait privilégié.

C. Limitations méthodologiques

L'étude porte sur un dataset de taille modeste ([~ 1700] images), ce qui limite la stabilité statistique des conclusions. Une validation croisée et la répétition des entraînements avec différentes *seeds* renforceraient la rigueur.

VIII. PERSPECTIVES

Les pistes prioritaires d'extension sont :

- **Suivi d'objets** (ByteTrack [8], DeepSORT) sur le flux vidéo pour stabiliser les détections ;
- **Reconnaissance de mots** via modélisation séquentielle (Transformer encoder sur séquence de lettres détectées) ;
- **Étude étendue de robustesse** : occlusions synthétiques, flou de mouvement, couleurs adverses ;
- **Distillation** de RF-DETR-S vers une architecture plus légère pour préserver la robustesse à coût réduit ;
- **Déploiement web complet** (FastAPI + Next.js + Docker) pour un service multi-utilisateur en production.

IX. CONCLUSION

Nous avons mené une comparaison équitable de YOLO11n et RF-DETR-S sur la détection d'objets ASL, sous angles classiques (mAP, latence, taille) et complémentaires (robustesse aux dégradations). En conditions nominales, les deux modèles affichent des performances quasi identiques (mAP@0.5 de 0.904 contre 0.901), avec un avantage net de YOLO11n sur les critères d'efficacité. L'étude de robustesse révèle néanmoins une fracture nette : RF-DETR-S préserve son mAP sous toutes les dégradations testées (chute moyenne 0.7 %) tandis que YOLO11n s'effondre sous bruit gaussien fort (chute de

76.8 %). Cette asymétrie illustre le contraste fondamental entre raisonnement local convolutif et attention globale transformeur, et fournit un guide concret de choix selon le contexte de déploiement. L'application web temps réel et le rapport complet sont publiés sur GitHub.

RÉFÉRENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once : Unified, real-time object detection," in *CVPR*, 2016.
- [2] G. Jocher *et al.*, "Ultralytics YOLO11," <https://docs.ultralytics.com/models/yolo11/>, 2024.
- [3] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *ECCV*, 2020.
- [4] Roboflow, "RF-DETR : A SOTA real-time object detection model," *ICLR*, 2026. <https://github.com/roboflow/rf-detr>
- [5] M. Oquab *et al.*, "DINOv2 : Learning robust visual features without supervision," *arXiv :2304.07193*, 2023.
- [6] D. Lee, "American Sign Language Letters Dataset," Roboflow Universe, 2020. <https://public.roboflow.com/object-detection/american-sign-language-letters>
- [7] A. Abid *et al.*, "Gradio : Hassle-free sharing and testing of ML models," <https://gradio.app>, 2019–2024.
- [8] Y. Zhang *et al.*, "ByteTrack : Multi-object tracking by associating every detection box," in *ECCV*, 2022.